

@adarschetan

SYSTEM DESIGN

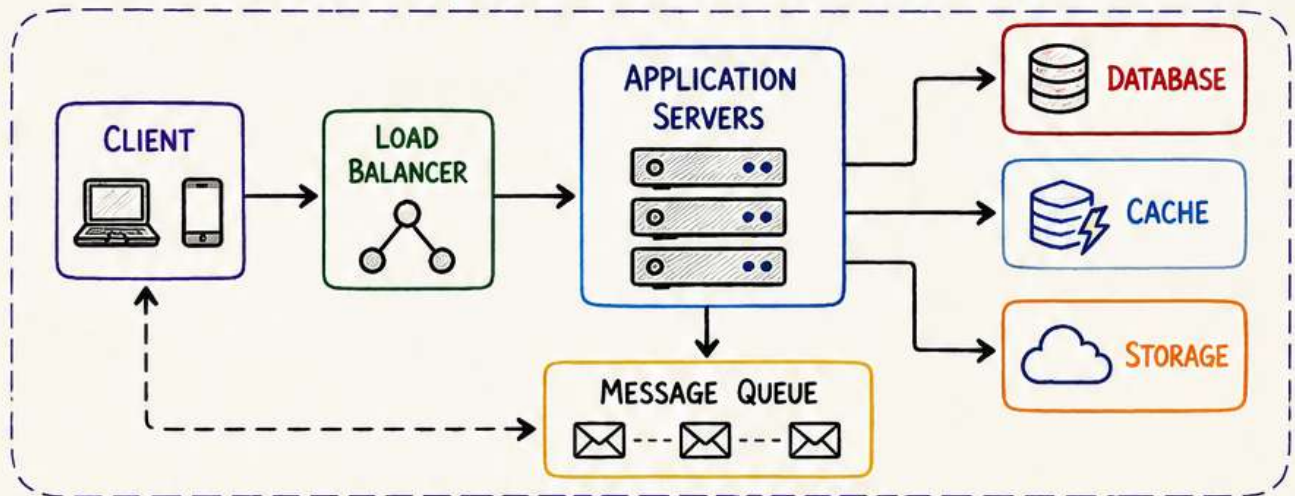
“ Good design
isn't just about
code. It's about
solving real-world
problems at scale.”

INTERVIEW

PATTERN

— GUIDE —

Master the patterns. Crack the interviews.



WHAT YOU'LL LEARN


SYSTEM DESIGN
FUNDAMENTALS


DESIGN PATTERNS
EXPLAINED


SCALABILITY &
PERFORMANCE


RELIABILITY &
AVAILABILITY


REAL WORLD
EXAMPLES

✓ Understand Patterns ✓ Apply the Right Solution ✓ Build Scalable Systems

THINK LIKE A DESIGNER.

SOLVE LIKE AN ENGINEER.

Do follow @AdarsChetan on LinkedIn & X
for more such amazing stuff!

DESIGN
BETTER
BUILD BIGGER.

TOP 10 MOST ASKED LLD DESIGN PATTERNS

"Good code is not just working code, it's well-designed code."

— EVERY SDE MUST KNOW —

Page 1 of 10

WHY DESIGN PATTERNS?



- ✓ Proven solutions to common design problems.
- ✓ Improve code reusability, maintainability and scalability.
- ✓ Reduce complexity and make code easy to understand.
- ✓ Help you think like a senior engineer.

★ REMEMBER:

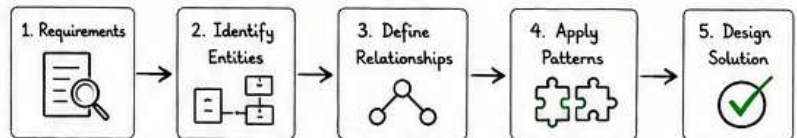
Patterns are not about copy-paste code. They are about solving **problems** in the **best** possible way.

HOW TO USE THIS GUIDE?



- Understand the problem.
- Identify where the pattern fits.
- Apply pattern and design the solution.
- Keep practicing with real-world examples.

DESIGN FLOW (HIGH LEVEL)

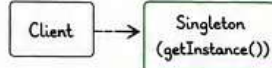


• THE TOP 10 DESIGN PATTERNS •

1 SINGLETON PATTERN

Ensures a class has only one instance and provides a global point of access to it.

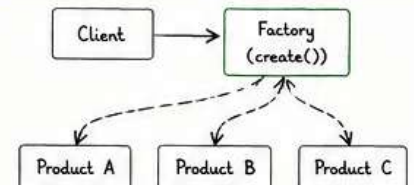
Use cases: Logger, Configuration Manager, Cache, DB Connection Pool



2 FACTORY PATTERN

Creates objects without exposing the creation logic to the client.

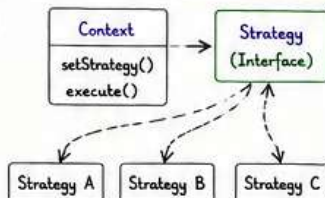
Use cases: Notification System, Payment Gateway, Vehicle Factory, UI Components



3 STRATEGY PATTERN

Defines a family of algorithms, encapsulates each one, and makes them interchangeable.

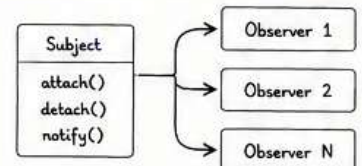
Use cases: Payment Methods, Sorting Algorithms, Pricing Engine, Discount Calculation



4 OBSERVER PATTERN

Defines a one-to-many dependency so that when one object changes state, all dependents are notified.

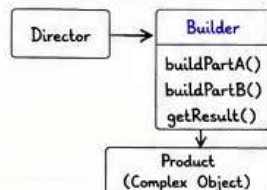
Use cases: Notification Service, Event Bus, Stock Price Updates, Chat Applications



5 BUILDER PATTERN

Separates construction of a complex object from its representation.

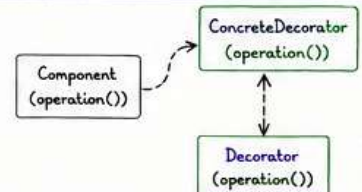
Use cases: User Builder, SQL Query Builder, HTTP Request Builder, Document Generator



6 DECORATOR PATTERN

Dynamically adds behavior / responsibilities to objects without modifying their code.

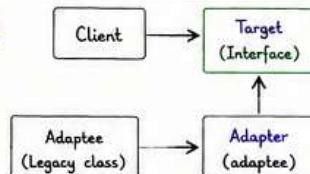
Use cases: Coffee Machine, Pizza Toppings, Logging, Compression, Encryption



7 ADAPTER PATTERN

Allows incompatible interfaces to work together.

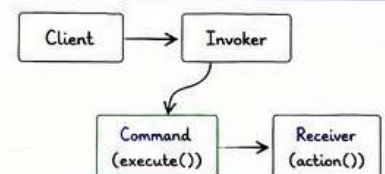
Use cases: Third-party APIs, Payment Providers, Legacy Systems, Database Migration



8 COMMAND PATTERN

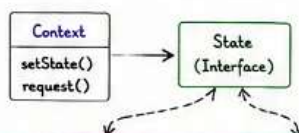
Encapsulates a request as an object, thereby letting you parameterize clients with different requests.

Use cases: Undo/Redo, Task Scheduler, Queue Processing, Remote Controls



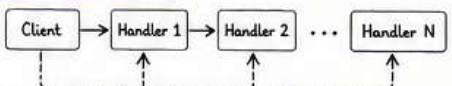
9 STATE PATTERN

Allows an object to alter its behavior when its internal state changes.



10 CHAIN OF RESPONSIBILITY PATTERN

Passes a request along a chain of handlers until one of them handles it.



★ Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff! ♥

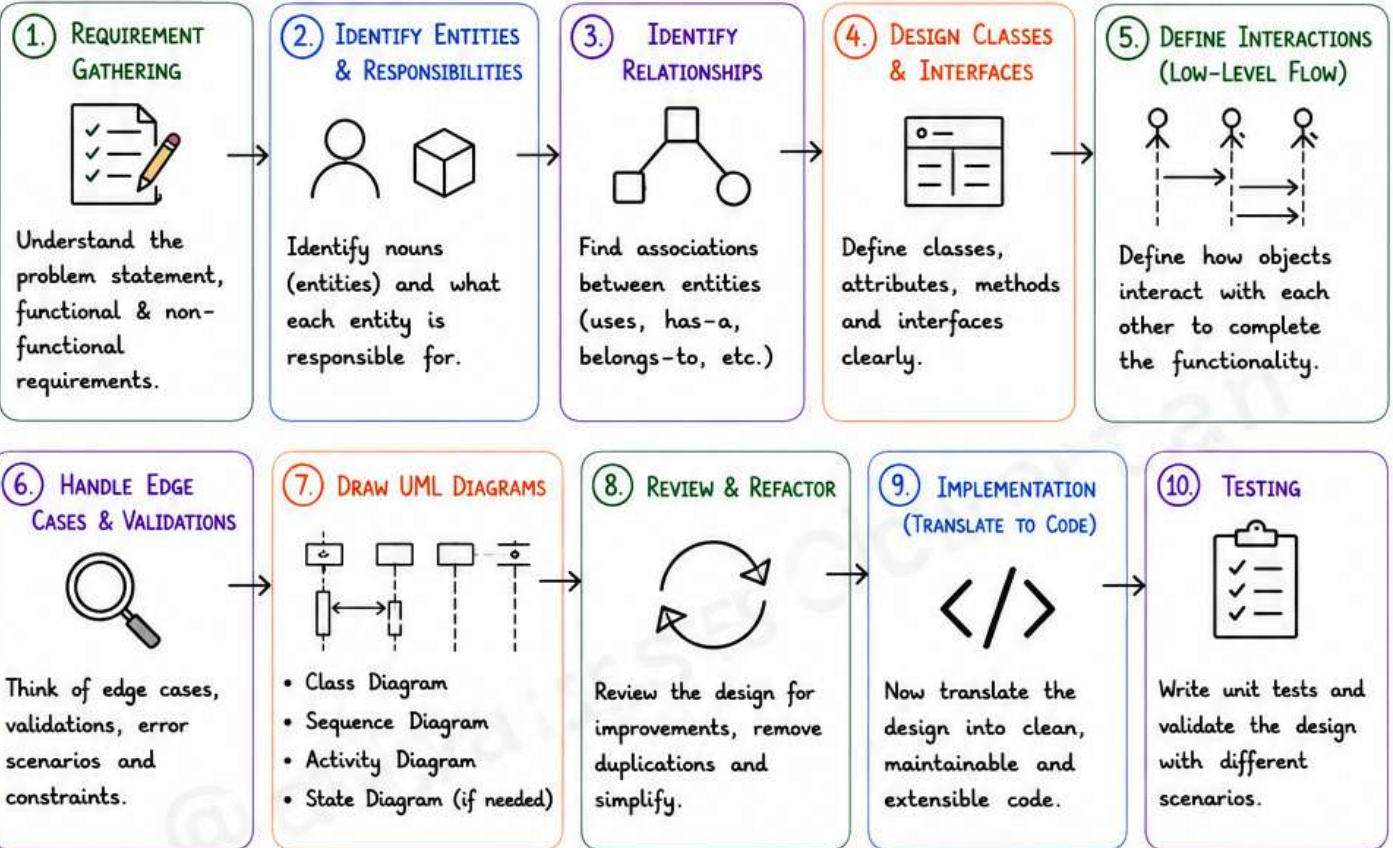


@adarschetan

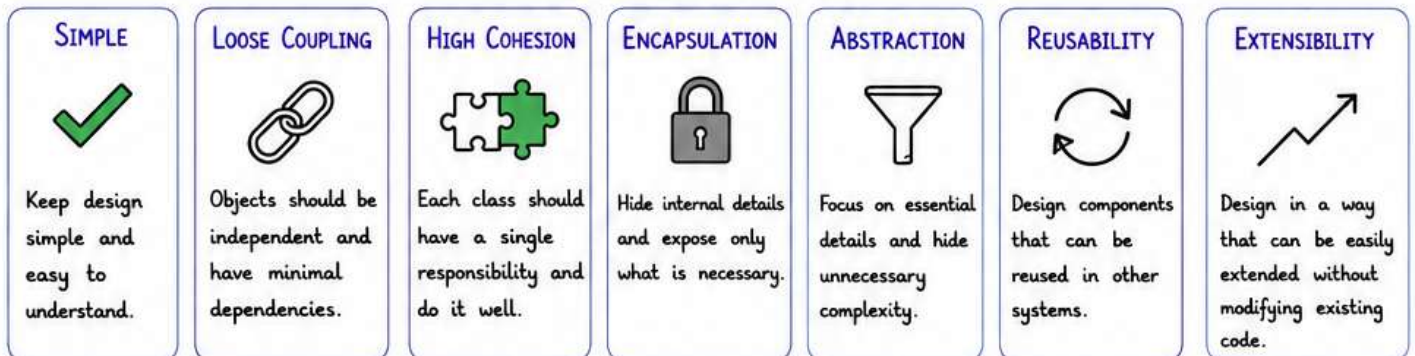
HOW TO SOLVE ANY LLD PROBLEM

Page 2 of 10

THE LLD DESIGN STEPS



KEY PRINCIPLES TO FOLLOW



CHOOSING OBJECTS - TIPS

→ Look for Nouns in the problem statement.

Example:

Problem: Design a Parking Lot System.

Nouns: ParkingLot, Vehicle, ParkingSpot, Ticket, Payment, Entrance, Exit.

- ✓ Don't blindly create classes. Think about responsibilities.
- ✓ One class = One responsibility (SRP).
- ✓ Avoid creating God classes.
- ✓ Prefer composition over inheritance.
- ✓ Program to interfaces, not implementations.

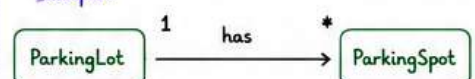
RELATIONSHIP TYPES

- Association (uses)
- Aggregation (has-a)
- Composition (strong has-a)
- Inheritance (is-a)
- Dependency (depends on)

COMMON MULTIPLICITIES

- 1 → Exactly one
- 0..1 → Zero or one
- * → Zero or more
- 1..* → One or more
- m..n → Between m and n
- n → Exactly n

Example:



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!



— @adarschetan

— UNDERSTAND THE DIFFERENCE —

Page 3 of 10

Aspect	High Level Design (HLD)	Low Level Design (LLD)
 Goal	• Overall system design and architecture.	• Detailed design of individual components and classes.
 Focus	• System components and their relationships.	• Classes, methods, attributes and their interactions.
 Level	• System-level (Macro view)	• Module-level (Micro view)
 Who Creates	• Solution Architect	• Developers / Designers
 What	• Modules, components, data flow, technologies.	• Classes, objects, methods, interfaces, relationships.
 Diagram	• Architecture Diagram, Component Diagram, Deployment Diagram.	• Class Diagram, Sequence Diagram, Activity Diagram, State Diagram.
 Example	• Designing the architecture of BookMyShow.	• Designing the Booking, Payment and Seat Management modules.

KEY TAKEAWAY



HLD is about the big picture.
LLD is about the details.

HLD answers "WHAT",
LLD answers "HOW".

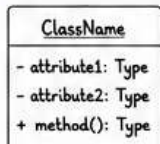
Example: Ride Booking App

HLD: User Service, Driver Service, Payment Service, Notification Service, DB, APIs.

LLD: User class, Driver class, Booking class, Payment class, methods, attributes, relations.

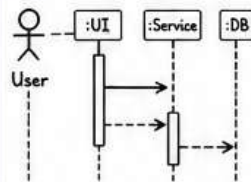
• TYPES OF DIAGRAMS IN LLD •

① CLASS DIAGRAM



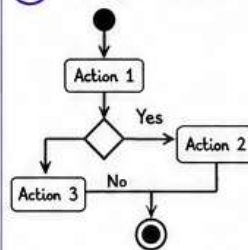
Shows classes, attributes, methods and relationships.

② SEQUENCE DIAGRAM



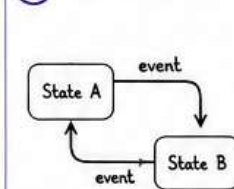
Shows object interactions in chronological order.

③ ACTIVITY DIAGRAM



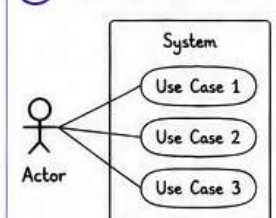
Shows workflow and business processes.

④ STATE DIAGRAM



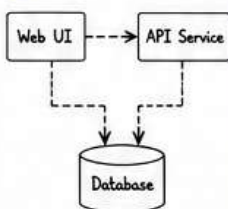
Shows states of an object and transitions between them.

⑤ USE CASE DIAGRAM



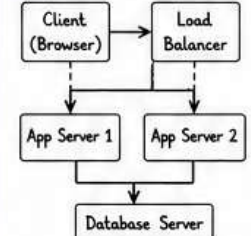
Shows functionalities from user's perspective.

⑥ COMPONENT DIAGRAM



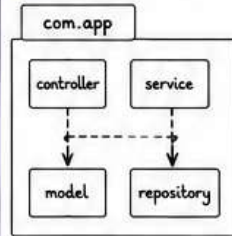
Shows system components and their dependencies.

⑦ DEPLOYMENT DIAGRAM



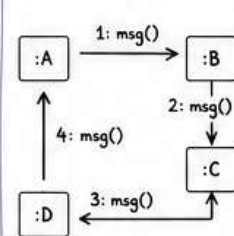
Shows physical deployment of components.

⑧ PACKAGE DIAGRAM



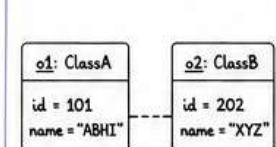
Shows organization of packages and dependencies.

⑨ COMMUNICATION DIAGRAM



Shows object interactions with messages.

⑩ OBJECT DIAGRAM



Shows runtime objects and their current state.

• TIPS FOR LLD INTERVIEW •

- ✓ Understand the requirements clearly.
- ✓ Identify entities, attributes and responsibilities.
- ✓ Choose the right objects and classes.
- ✓ Apply SOLID principles.
- ✓ Use design patterns where required.

- ✓ Draw diagrams to explain your design.
- ✓ Handle edge cases and exceptions.
- ✓ Keep the design extensible and maintainable.
- ✓ Communicate your thought process.
- ✓ Review and refine the design.

Remember:

Good design is simple, flexible and easy to extend.



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!



— @adarschetan

WHAT IS OOP?

Object Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data (attributes) and code (methods).

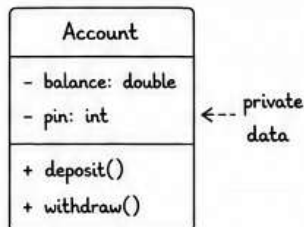
BENEFITS OF OOP

- ✓ Modularity
- ✓ Reusability
- ✓ Maintainability
- ✓ Scalability
- ✓ Flexibility
- ✓ Real-world modeling

THE 4 PILLARS OF OOP

1 ENCAPSULATION

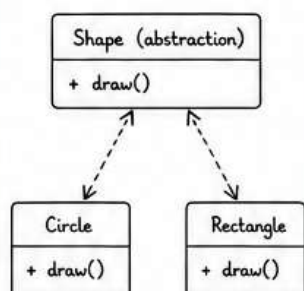
Bundling data (attributes) and methods that operate on the data into a single unit (class). Restrict direct access to some components.



Example: Bank Account

2 ABSTRACTION

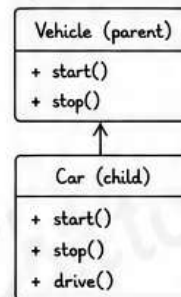
Hiding unnecessary details and showing only essential features to the user.



Example: Shape, DB Connection

3 INHERITANCE

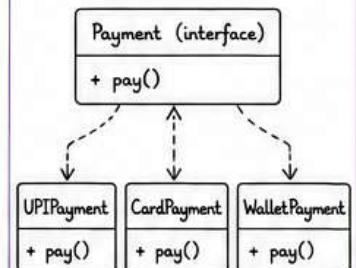
Creating a new class from an existing class. The new class (child) inherits properties and behaviors of the existing class (parent).



Example: Car is a Vehicle

4 POLYMORPHISM

The ability of an object to take many forms. Same interface, different implementations.

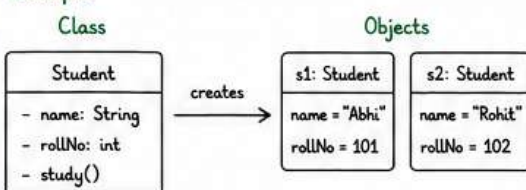


Example: Multiple Payment Methods

CLASS VS OBJECT

- **Class:** Blueprint or template for creating objects.
- **Object:** Instance of a class.

Example:



RELATIONSHIPS BETWEEN CLASSES

- Association (uses)
- Aggregation (has-a)
- Composition (strong has-a)
- Inheritance (is-a)
- Dependency (depends on)

OOP PRINCIPLE

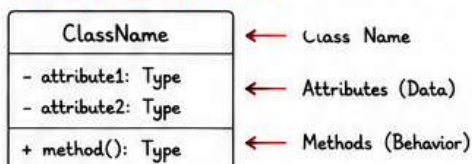
Treat objects as real-world entities. Focus on what they do, not how they do it internally.



ACCESS MODIFIERS (VISIBILITY)

Modifier	Same Class	Same Package	Sub Class (Different Package)	Other Package (Non-Sub Class)	Description
public	✓	✓	✓	✓	Accessible everywhere.
protected	✓	✓	✓	✗	Accessible in same package and subclasses.
default (package-private)	✓	✓	✗	✗	Accessible in the same package only.
private	✓	✗	✗	✗	Accessible within the class only.

UML CLASS NOTATION



BEST PRACTICES

- ✓ Follow SOLID Principles.
- ✓ Keep classes small and focused.
- ✓ Prefer composition over inheritance.
- ✓ Program to interfaces, not implementations.
- ✓ Write clean, readable and maintainable code.

REMEMBER:

Good OOP design makes your code easy to understand, change and extend.





WHAT IS SOLID?

SOLID is a set of five design principles that help us create code that is easier to understand, maintain and extend.

WHY IT MATTERS?

- ✓ Reduces coupling between classes
- ✓ Improves code readability
- ✓ Enhances reusability and testability
- ✓ Makes the system easier to change and extend

THE 5 SOLID PRINCIPLES

1 SINGLE RESPONSIBILITY PRINCIPLE (SRP)

A class should have only one reason to change, meaning it should have only one responsibility.

Example:

```
class Invoice {
    calculateTotal()
}

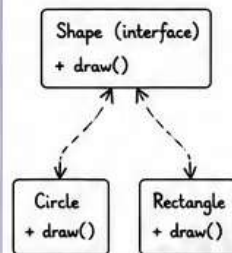
class InvoicePrinter {
    printInvoice()
}
```

Each class does one job.

2 OPEN/CLOSED PRINCIPLE (OCP)

Software entities should be open for extension but closed for modification.

Example:

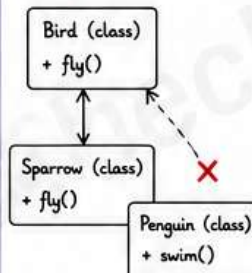


We can add new shapes without changing existing code.

3 LISKOV SUBSTITUTION PRINCIPLE (LSP)

Objects of a superclass should be replaceable with objects of its subclasses without breaking the application.

Example:

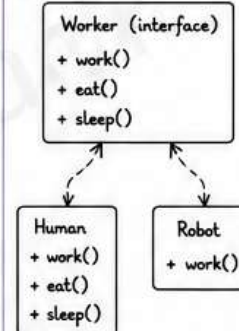


Penguin is not a Bird.

4 INTERFACE SEGREGATION PRINCIPLE (ISP)

A class should not be forced to implement interfaces it does not use.

Example:

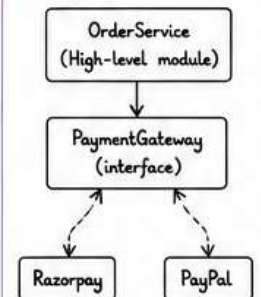


Keep interfaces small and specific.

5 DEPENDENCY INVERSION PRINCIPLE (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Example:



Depend on abstraction (interface), not on concrete classes.

COMMON OOP CONCEPTS IN LLD

CLASS	OBJECT	ATTRIBUTE	METHOD	CONSTRUCTOR	ENCAPSULATION	ABSTRACTION	INHERITANCE	POLYMORPHISM
Blueprint for creating objects.	Instance of a class.	Properties of a class.	Functions or behaviors.	Special method to initialize objects.	Binding data and methods together & restricting direct access.	Showing only essential details, hiding implementation.	Deriving new class from existing class.	One interface, many implementations.
Example: class Car { }	Example: Car c1 = new Car();	Example: String name;	Example: void start();	Example: Car() { }	Example: private String name;	Example: interface Shape { }	Example: class Bike extends Vehicle { }	Example: Shape s = new Circle(); s.draw();

WHEN TO USE OOP IN LLD?

- ✓ When the system has many related entities.
- ✓ When the system is expected to grow and change.
- ✓ When we need reusability and extensibility.
- ✓ When modeling real-world scenarios.

Real World Problem

Identify Objects & Responsibilities

Design Classes & Relationships

Good Design

Remember: OOP is a tool, not a rule. Use it wisely where it adds value.



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!



@adarschetan



WHAT IS UML?

UML (Unified Modeling Language) is a standard visual language used to model software systems. It helps in designing, visualizing and documenting systems.

WHY UML?

- ✓ Provides a common way to visualize systems
- ✓ Improves communication among team members
- ✓ Helps in understanding complex systems
- ✓ Acts as a blueprint for developers

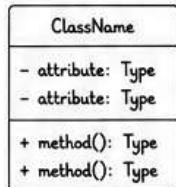
TIP:

You don't need to use all diagrams every time. Use the ones that add the most value. ★

• TYPES OF UML DIAGRAMS •

① CLASS DIAGRAM

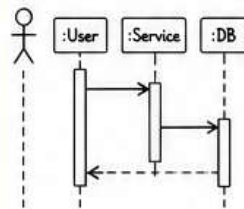
Shows the static structure of the system - classes, attributes, methods and relationships.



Use for:
Modeling the structure of classes and their relationships.

② SEQUENCE DIAGRAM

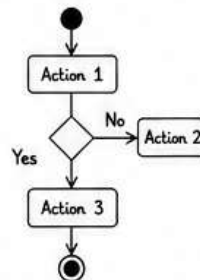
Shows object interactions in a time sequence.



Use for:
Understanding how objects interact with each other over time.

③ ACTIVITY DIAGRAM

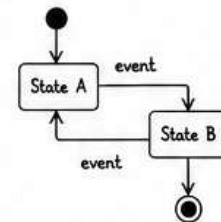
Shows the flow of activities or business processes.



Use for:
Modeling workflows and business processes.

④ STATE DIAGRAM

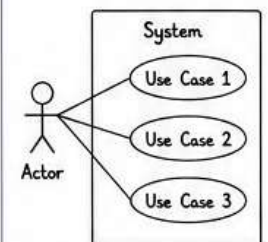
Shows different states of an object and transitions between them.



Use for:
Modeling state transitions of objects.

⑤ USE CASE DIAGRAM

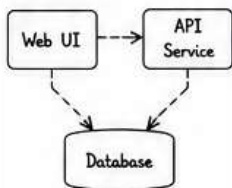
Shows system functionality from the user's perspective.



Use for:
Capturing requirements and system scope.

⑥ COMPONENT DIAGRAM

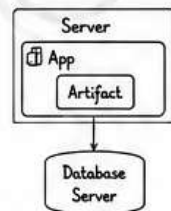
Shows the components of the system and their dependencies.



Use for:
Modeling system components and their dependencies.

⑦ DEPLOYMENT DIAGRAM

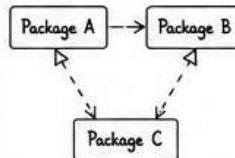
Shows the physical deployment of artifacts on hardware nodes.



Use for:
Modeling deployment architecture.

⑧ PACKAGE DIAGRAM

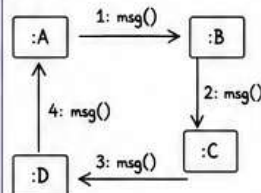
Shows how packages are organized and depend on each other.



Use for:
Organizing model elements into packages.

⑨ COMMUNICATION DIAGRAM

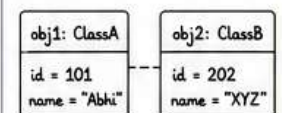
Shows object interactions and the links between them.



Use for:
Alternative to sequence diagram (focus on links).

⑩ OBJECT DIAGRAM

Shows the instances of classes at a specific point in time.



Use for:
Showing snapshot of objects and their values.

RELATIONSHIP NOTATIONS (IN CLASS DIAGRAM)

Association (uses)	A —> B
Aggregation (has-a)	A ◊— B
Composition (strong has-a)	A ◆— B
Inheritance (is-a)	A — > B
Dependency (depends on)	A -.-> B

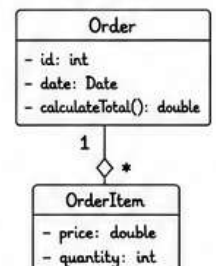
VISIBILITY NOTATIONS

+ Public	- Accessible everywhere
- Private	- Accessible within class
# Protected	- Accessible in class & subclasses
~ Package	- Accessible within package

MULTIPLICITY NOTATIONS

1	Exactly one
0..1	Zero or one
*	Zero or more
1..*	One or more
m..n	Between m and n
n	Exactly n

EXAMPLE



WHEN TO USE UML DIAGRAM?

- ✓ When the system has many related entities.
- ✓ When the system is expected to grow and change.
- ✓ When we need reusability and extensibility.
- ✓ When modeling real-world scenarios.

Real World Problem

Identify Objects & Responsibilities

Design Classes & Relationships

Good Design

Remember: UML is a tool, not a rule. Use it wisely where it adds value. ★



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!



— @adarschetan

WHAT ARE DESIGN PATTERNS?



Design Patterns are general, reusable solutions to commonly occurring problems in software design. They are not finished designs - they are templates or blueprints that you can adapt to solve problems in your code.

TYPES OF DESIGN PATTERNS (GANG OF FOUR)

- **Creational Patterns** — Object creation mechanisms.
- **Structural Patterns** — Object composition & relationships.
- **Behavioral Patterns** — Communication between objects.

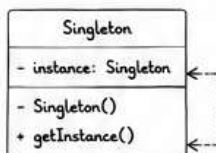
TOP 10 MOST ASKED LLD DESIGN PATTERNS

1 SINGLETON PATTERN

Problem: Ensure a class has only one instance and provide a global point of access to it.

Idea: Make constructor private and provide a static method.

Example: Logger, Config Manager, DB Connection

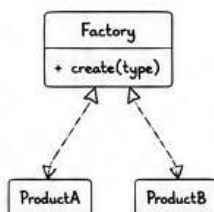


2 FACTORY PATTERN

Problem: Create objects without exposing the instantiation logic.

Idea: Use a factory class or method to create objects.

Example: Payment Gateway, Notification, Vehicle Factory

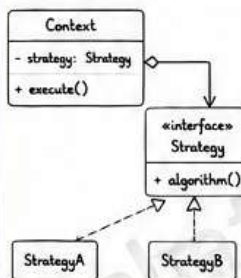


3 STRATEGY PATTERN

Problem: Define a family of algorithms, encapsulate each one, and make them interchangeable.

Idea: Create algorithm interfaces and concrete strategies.

Example: Payment Methods, Sorting, Discount Calculation

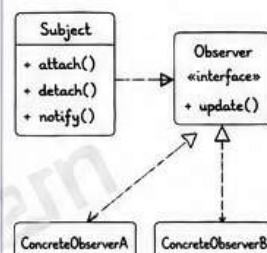


4 OBSERVER PATTERN

Problem: Define a one-to-many dependency so that when one object changes state, all its dependents are notified.

Idea: Subject notifies all observers about state changes.

Example: Notification Service, Event Bus, Stock Updates

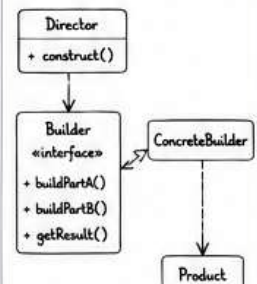


5 BUILDER PATTERN

Problem: Construct complex objects step by step.

Idea: Separate construction of a complex object from its representation.

Example: User Builder, HTTP Request, SQL Builder

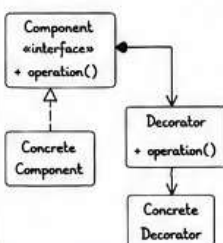


6 DECORATOR PATTERN

Problem: Add new behavior to an object dynamically without altering its structure.

Idea: Wrap the object with decorators that add responsibilities.

Example: Coffee Machine, Pizza Toppings, Logging

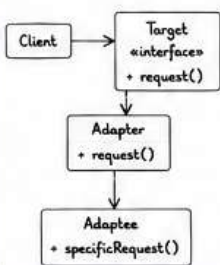


7 ADAPTER PATTERN

Problem: Make incompatible interfaces work together.

Idea: Create an adapter class that converts one interface into another.

Example: Third-party APIs, Payment Providers, Legacy Systems

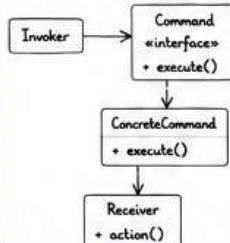


8 COMMAND PATTERN

Problem: Encapsulate a request as an object, thereby letting you parameterize clients with different requests.

Idea: Represent requests as objects and execute them.

Example: Undo/Redo, Task Scheduler, Queue Processing

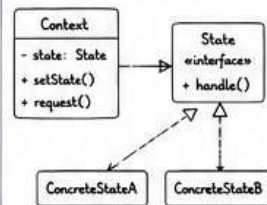


9 STATE PATTERN

Problem: Allow an object to alter its behavior when its internal state changes.

Idea: Create state classes and change behavior based on state.

Example: ATM Machine, Elevator, Vending Machine

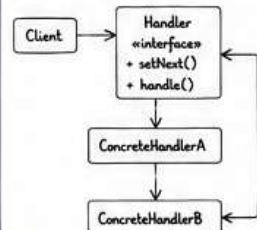


10 CHAIN OF RESPONSIBILITY PATTERN

Problem: Avoid coupling the sender of a request to its receiver by giving more than one object a chance.

Idea: Pass the request along a chain of handlers.

Example: Authentication, Logging, Approval Workflow



PATTERN SELECTION GUIDE

Situation	Use Pattern
Need only one instance	Singleton
Object creation logic complex	Factory
Need multiple algorithms	Strategy
One-to-many dependency	Observer
Complex object construction	Builder
Add behavior dynamically	Decorator
Incompatible interfaces	Adapter
Encapsulate a request	Command
Object behavior changes	State
Pass request along a chain	Chain of Responsibility

BEST PRACTICES

- ✓ Prefer composition over inheritance.
- ✓ Program to interfaces, not implementations.
- ✓ Keep classes small and focused.
- ✓ Follow SOLID principles.
- ✓ Choose the right pattern, not any pattern.
- ✓ Don't overuse patterns.
- ✓ Document pattern usage with examples.

REMEMBER:

Design patterns are not magic. They are experience from other developers. Use the right pattern at the right time to build scalable and maintainable systems.



WHY USE DESIGN PATTERNS?



- ✓ Proven solutions to common problems.
- ✓ Improve code reusability and readability.
- ✓ Make code easier to understand and maintain.
- ✓ Speed up development and reduce bugs.
- ✓ Enable communication among developers.

PATTERN CATEGORIES (GANG OF FOUR)

- **Creational Patterns** — How objects are created.
- **Structural Patterns** — How classes/objects are composed.
- **Behavioral Patterns** — How objects communicate.

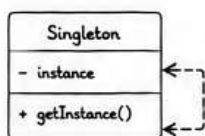
TOP 10 MOST ASKED LLD DESIGN PATTERNS (SUMMARY)

① SINGLETON PATTERN

Problem: Ensure a class has only one instance and provide a global point of access to it.

Example:

Logger, Config Manager, DB Connection

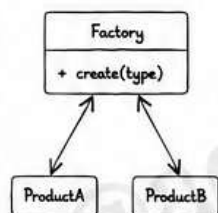


② FACTORY PATTERN

Problem: Create objects without exposing the instantiation logic.

Example:

Payment Gateway, Notification, Vehicle Factory

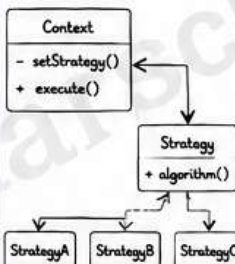


③ STRATEGY PATTERN

Problem: Define a family of algorithms, encapsulate each one, and make them interchangeable.

Example:

Payment Methods, Sorting Algorithms, Discount Calculation

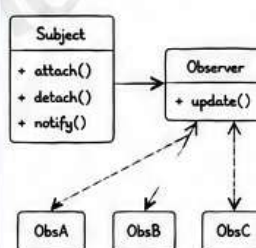


④ OBSERVER PATTERN

Problem: Define a one-to-many dependency so that when one object changes state, all its dependents are notified.

Example:

Notification Service, Event Bus, Stock Price Updates

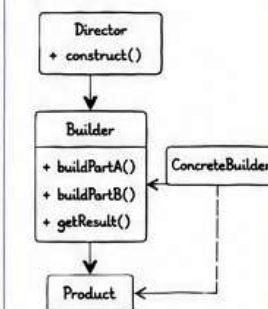


⑤ BUILDER PATTERN

Problem: Construct complex objects step by step. Separate construction from representation.

Example:

User Builder, HTTP Request, SQL Builder

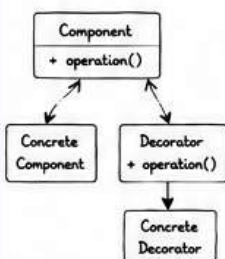


⑥ DECORATOR PATTERN

Problem: Add new behavior to an object dynamically without altering its structure.

Example:

Coffee Machine, Pizza Toppings, Logging

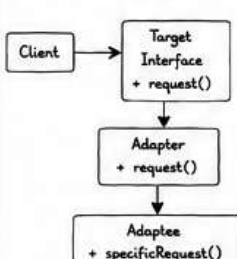


⑦ ADAPTER PATTERN

Problem: Make incompatible interfaces work together.

Example:

Third-party APIs, Payment Providers, Legacy Systems

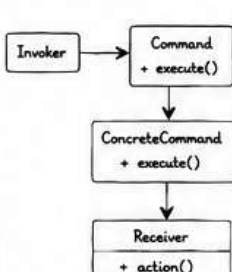


⑧ COMMAND PATTERN

Problem: Encapsulate a request as an object, thereby letting you parameterize clients with different requests.

Example:

Undo/Redo, Task Scheduler, Queue Processing

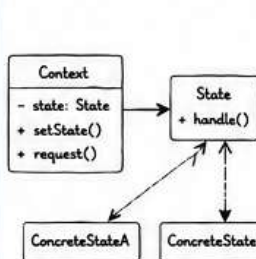


⑨ STATE PATTERN

Problem: Allow an object to alter its behavior when its internal state changes.

Example:

ATM Machine, Elevator, Vending Machine

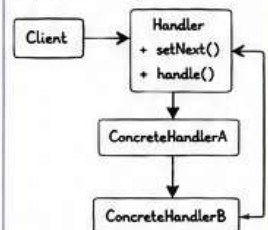


⑩ CHAIN OF RESPONSIBILITY PATTERN

Problem: Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request.

Example:

Authentication, Logging, Approval Workflow



PATTERN SELECTION GUIDE

Situation	Use Pattern
Need only one instance	Singleton
Create objects based on type	Factory
Multiple algorithms, same work	Strategy
One-to-many notifications	Observer
Complex object creation	Builder
Add responsibilities dynamically	Decorator
Incompatible interfaces	Adapter
Request as an object	Command
Behavior changes with state	State
Pass request along a chain	Chain of Responsibility

BEST PRACTICES

- ✓ Prefer composition over inheritance.
- ✓ Program to interfaces, not implementations.
- ✓ Keep classes small and focused.
- ✓ Follow SOLID principles.
- ✓ Choose the right pattern, not any pattern.
- ✓ Don't overuse patterns.
- ✓ Understand the problem before the pattern.
- ✓ Document pattern usage with examples.

REMEMBER:

- ★ Patterns are tools, not rules.
- ★ Right pattern in the right time makes your design clean and powerful.



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!



— @adarschetan